Reconstruct Bst

AlgoExpert · Medium · Binary Search Trees

The problem

- Rebuild a BST from its pre-order traversal (duplicates go right) using value bounds.

Intuition

- In pre-order, the root comes first and everything smaller goes left, so we build with value bounds.

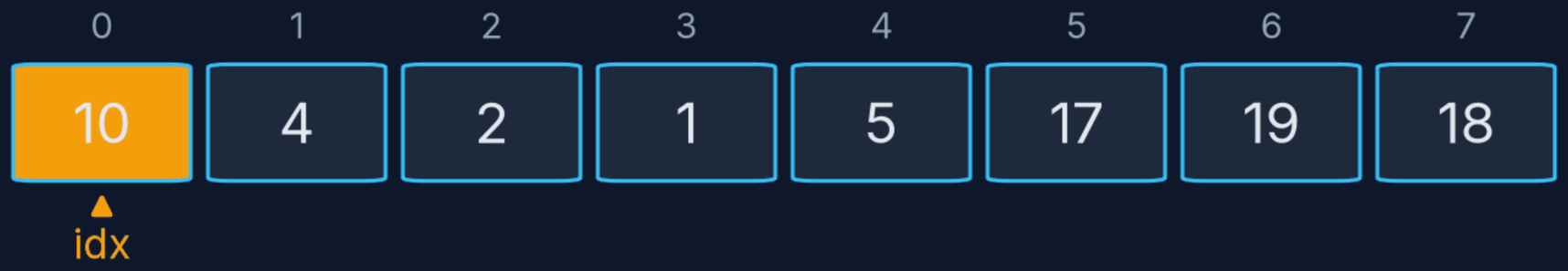
Brute force vs optimal

- Insert each value into a BST → $O(n^2)$ worst, $O(n \log n)$ average
- For each node, scan for where the right subtree starts → $O(n^2)$
- Bounds recursion with a shared index → $O(n)$

Approach

- 1. Read values in order
- 2. Each recursion accepts values within a lower and upper bound
- 3. Root first, then left with an upper bound, then right

Worked example

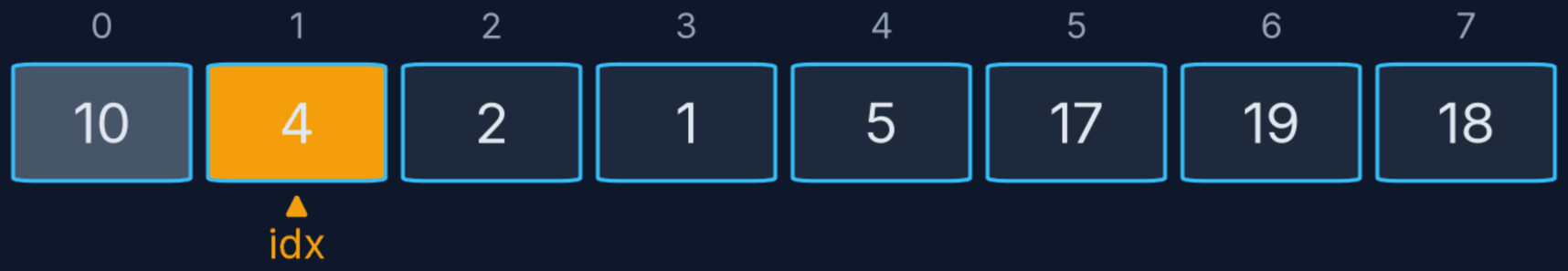




`bounds [-∞, ∞)`

10 fits bounds → new node

Worked example

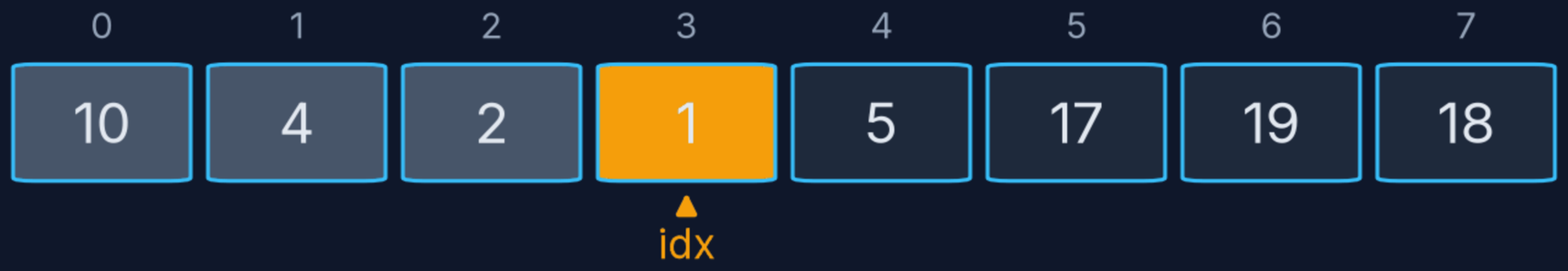




`bounds [-∞, 10)`

4 fits bounds → new node

Worked example

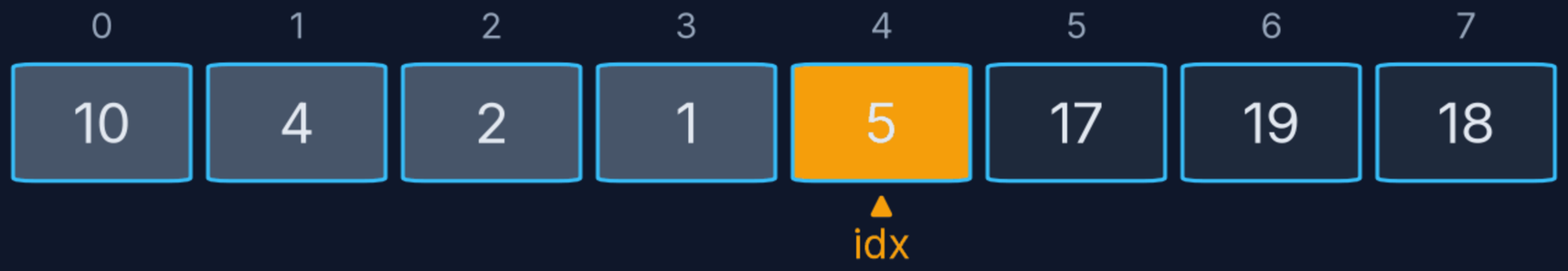




`bounds [-∞, 2)`

1 fits bounds → new node

Worked example

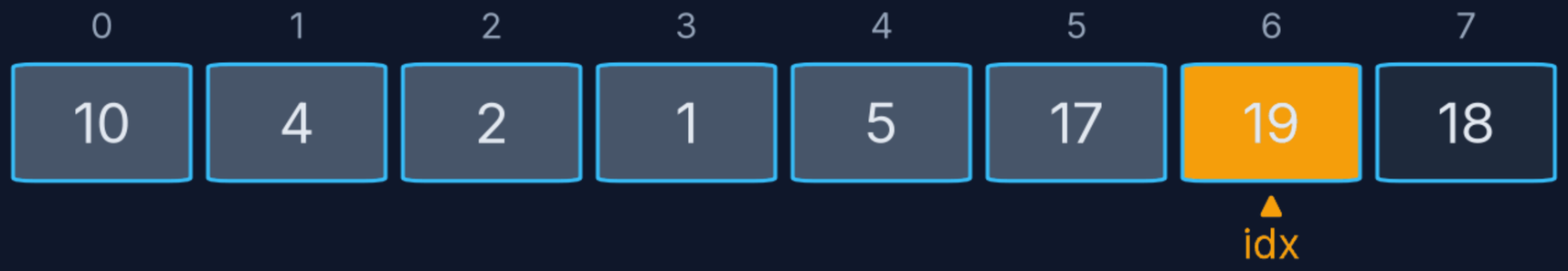


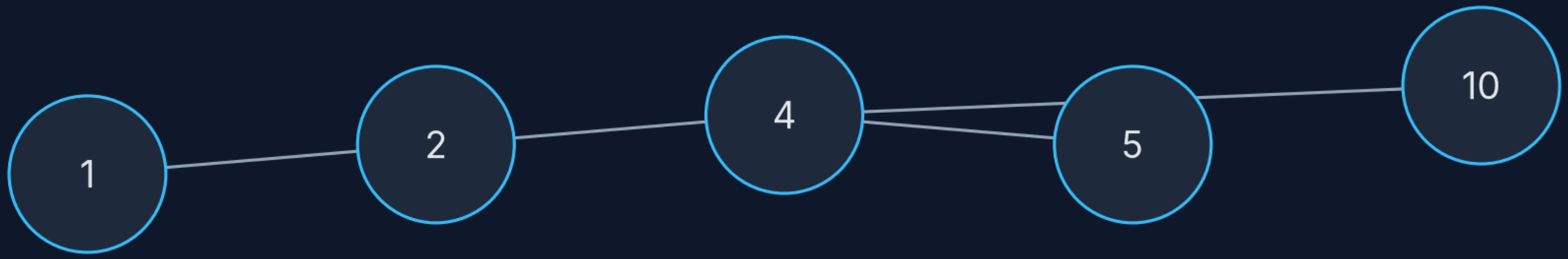


`bounds [4, 10)`

5 fits bounds → new node

Worked example

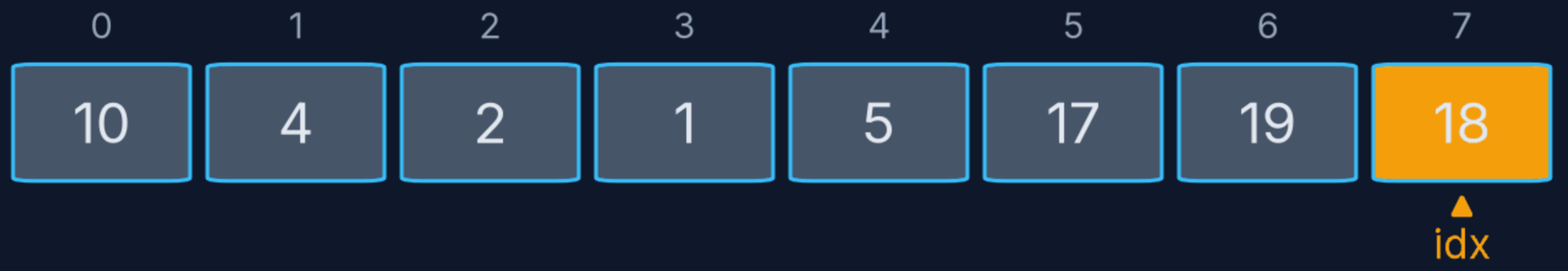


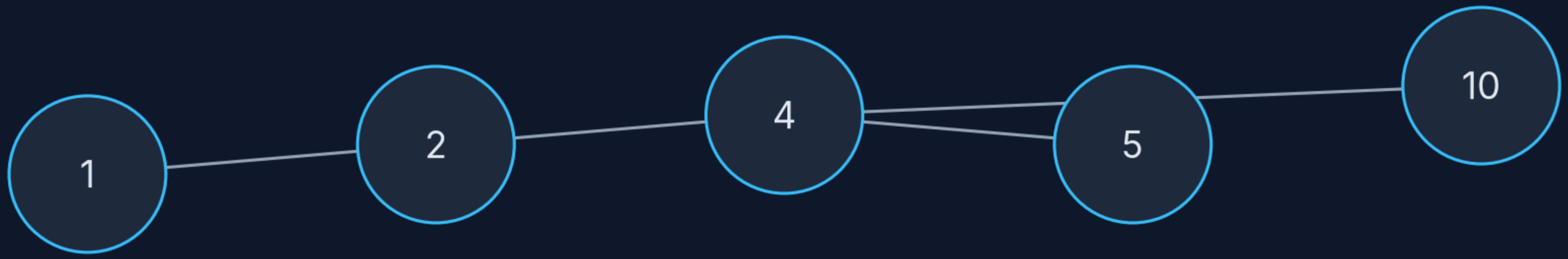


`bounds [17, ∞)`

19 fits bounds → new node

Worked example





`bounds [17, 19)`

18 fits bounds → new node

Java solution

```java
public class ReconstructBst {
    private static int idx;

    public static synchronized BST reconstructBst(List<Integer> pre) {
        idx = 0;
        return build(pre, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private static BST build(List<Integer> pre, long lo, long hi) {
        if (idx == pre.size()) return null;
        int v = pre.get(idx);
        if (v < lo || v >= hi) return null;
        idx++;
        BST n = new BST(v);
        n.left = build(pre, lo, v);
        n.right = build(pre, v, hi);
        return n;
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Edge cases & pitfalls

- Duplicates go right: use [lo, hi) bounds
- Use long bounds to avoid overflow
- Reset the shared index per call

Interview tips & follow-ups

- Explain why the index only moves forward
- Follow-up: from post-order; from in-order + pre-order (general tree)

Recap

- In pre-order, the root comes first and everything smaller goes left, so we build with value bounds.
- Complexity: O(n)